

PROJETO DE CPE

computação para engenharia
professor Wesin Ribeiro

```
e {  
header1.css('padding-top', '' + h  
  
window).scrollTop() > header2_in  
parseInt(header2.css('padding-to  
header2.css('padding-top', '' +  
  
( 'padding-top', '' + h
```

01

INTEGRANTES



**GISELE
CRISTINE**



**MIKAEL
VITOR**



**ALEX
LANDIM**



**BRUNO
NICHOLAS**



**GIOVANA
LOURES**

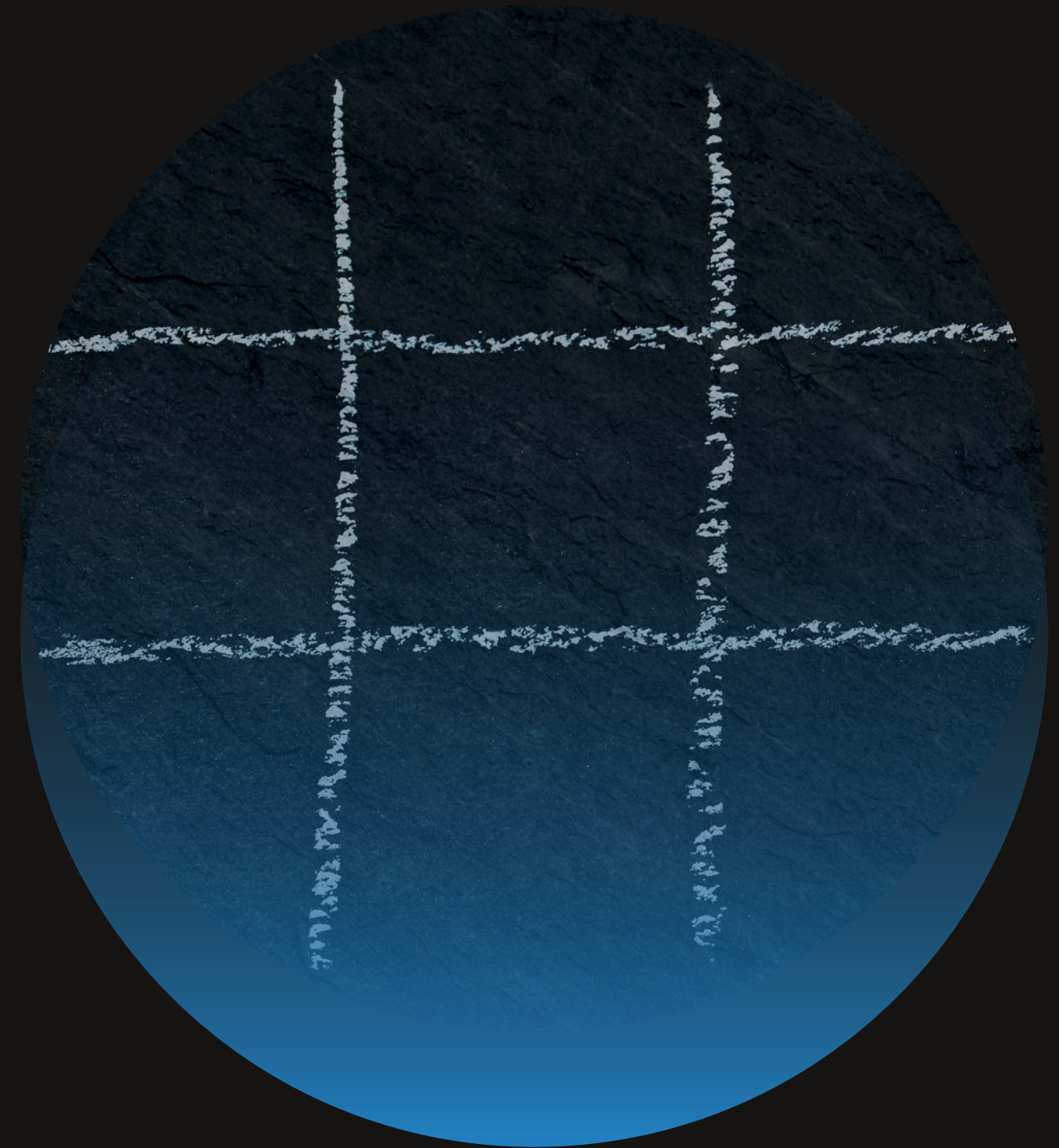
02 INTRODUÇÃO

O JOGO DA VELHA

O tradicional jogo da velha é um jogo de tabuleiro simples para dois jogadores, que alternam turnos para marcar "X" ou "O" em uma grade de 3x3. O objetivo é formar uma linha com três símbolos iguais, seja na horizontal, vertical ou diagonal. Se todas as casas forem preenchidas sem um vencedor, o jogo termina em empate.

LINGUAGEM C++

O projeto consiste em uma adaptação do tradicional jogo da velha de tabuleiro para a linguagem C++, para isso foram utilizados todos os conceitos aprendidos em sala.



03 PASSOS PARA O CODIGO

- 01 O primeiro passo foi a declaração das bibliotecas
- 02 Criação de struct para armazenar informações sobre o jogo
- 03 Inicializar variáveis – Define o tabuleiro vazio e configura os jogadores.
- 04 Imprimir tabuleiro – Exibe a matriz do jogo no console.
- 05 Verificar vitória – Confere se há três símbolos iguais em uma linha, coluna ou diagonal
- 06 Verificar empate – Retorna verdadeiro se todas as casas estiverem preenchidas sem vencedor

```
para inicializar
umessarujogu(Jogodavelha
Jogodavelha.tabuleiro = new
//posições iniciadas
Jogodavelha.rodada = 'X'; //
Jogodavelha.pontuacaoDoX = 0;
Jogodavelha.pontuacaoDoO = 0;
}

/procedimento que libera memória
oid liberarJogo(Jogodavelha& Jogo
delete[] Jogodavelha.tabuleiro

/procedimento que imprime o tab
printabuleiro(const Jogoda
for (int i = 0; i < 9; ++i
```

04 PASSOS PARA O CODIGO

- 07 Guardar pontuação – Salva o placar do vencedor em um arquivo.
- 08 Rodar o jogo – Gerencia turnos, entrada de jogadores e verifica condições de vitória/empate.
- 09 Liberar memória – Libera espaço alocado dinamicamente, se necessário
- 10 Função main – Inicializa o jogo, chama a função principal e exibe os resultados.

```
para inicializar
umessaruJogu(Jogodavelha
Jogodavelha.tabuleiro = new
//posições iniciadas
Jogodavelha.rodada = 'X'; //
Jogodavelha.pontuacaoDoX = 0;
Jogodavelha.pontuacaoDoO = 0;
}

/procedimento que libera memória
oid liberarJogo(Jogodavelha& Jogo
delete[] Jogodavelha.tabuleiro

/procedimento que imprime o tab
printabuleiro(const Jogoda
for (int i = 0; i < 9; ++i)
```


05 EXPLICAÇÃO DO CÓDIGO

Primeiras linhas

as primeiras linhas são a declaração das bibliotecas. A biblioteca padrão <iostream> e a biblioteca <fstream> que possui a função ofstream usada no código para escrever em arquivos as pontuações dos jogadores

Inicializar variáveis

O procedimento "kumessarujogo" inicializa o tabuleiro e o jogo, usando como parâmetro o tipo da struct. Esta função inicializa também as outras informações da struct a fim e preparar e dar início ao jogo. "kumessarujogo" usa um ponteiro para alocação dinâmica de memória no momento que inicializa o tabuleiro a fim de utilizar memória de forma segura.



06 EXPLICAÇÃO DO CÓDIGO

Struct

A partir da linha 6, nós declaramos a struct "Jogodavelha" que armazena dados usados no decorrer do jogo, como um ponteiro para a execução do tabuleiro, uma variável para a alternância dos jogadores e as variáveis que armazenam a pontuação dos jogadores

```
6  ✓ struct Jogodavelha {  
7      char* tabuleiro; //ponteiro  
8      char rodada; // X ou O  
9      int pontuacaoDoX; // pontuação do jogador X  
10     int pontuacaoDoO; // inicialização do O  
11     };
```

07 EXPLICAÇÃO DO CÓDIGO

Procedimentos

O procedimento "printabuleiros" imprime o tabuleiro usando uma lógica que funciona da seguinte forma: Entre os números 0 e 9 existem 3 múltiplos de 3, o próprio 3, o 6 e o 9. Isso significa que a cada 3 unidades existe um múltiplo de 3, o que faz todo sentido logicamente. Como o tabuleiro tem 3 "espaços" em cada linha, sempre que o iterador chegar a um múltiplo de 3, ele imprime uma quebra de linha e isso gera 3 linhas com 3 espaços cada, dando corpo ao tabuleiro. O que separa os espaços é um " | " que é impresso quando o iterador percorre valores que não são múltiplos de 3.

```
//procedimento que imprime o tabuleiro do jogo
void printabuleiro(const Jogodavelha& Jogodavelha) {
    for (int i = 0; i < 9; ++i)
    {
        cout << Jogodavelha.tabuleiro[i];
        if (i % 3 == 2) {
            cout << endl;
        } else{cout << " | ";}
        if (i % 3 == 2 && i < 8) cout << "-----\n";
    } // tem que ser múltiplo de 3 pois o tabuleiro é de 3x3. então se for múltiplo de 3 é pq está no final da linha por
```


08 EXPLICAÇÃO DO CÓDIGO

Verificar vitoria

A função "verificaVitoria" é responsável por checar se um dos jogadores ganhou a partida. Ela é uma função do tipo booleano, ou seja, retorna um valor verdadeiro ou falso, nesse caso um valor verdadeiro caso alguém tenha ganhado e falso caso o contrário.

Esta função possui um comando um pouco peculiar. Quando declaramos o ponteiro para o tabuleiro, ele foi declarado como um array de 9 caracteres, no entanto, em certo momento do código ele precisa ser interpretado como uma matriz 3x3. A linha 41 possui comandos para reinterpretar o tabuleiro como uma matriz `[*t][3]` onde o ponteiro `t` é definido implicitamente no laço da linha 44 fazendo com que o tabuleiro seja interpretado como uma matriz 3x3.

Verificar empate

A Função "verificarEmpate" percorre o tabuleiro e finaliza quando não houver mais espaços vazios.

09 EXPLICAÇÃO DO CÓDIGO

Guardar pontuacao

procedimento que escreve a pontuação dos jogadores no arquivo. O procedimento usa a função ofstream para escrever no arquivo criado (pontuação.txt) e depois salva e fecha o mesmo

Rodar jogo

O procedimento "jogujogo" chama algumas funções já citadas e roda o jogo, de forma que integra as escolhas com o tabuleiro e as verificações de vitória e empate. A função main chama as funções escritas e encerra o código ao fim

```
...ção main a l  
main() {  
    //chamada das funç  
    Jogodavelha Jogodave  
    kumessarujogu(Jogodav  
    jogujogu(Jogodavelha  
    liberarJogo(Jogodave  
    return 0;  
}
```

CONCEITOS ABORDADOS



- ▶ **STRUCT**
- ▶ **PONTEIROS**
- ▶ **VETORES
(ARRAYS)**
- ▶ **FUNCAO**
- ▶ **MATRIZES**

Uma struct em C++ é uma estrutura que agrupa variáveis de diferentes tipos sob um mesmo nome. É útil para representar objetos mais complexos, como um jogador com nome, pontuação e nível.

Um ponteiro é uma variável que armazena o endereço de outra variável na memória. Ele permite manipulação eficiente de dados e é usado em alocação dinâmica, arrays e passagem de parâmetros por referência.

Vetores e arrays armazenam múltiplos elementos do mesmo tipo em posições contíguas da memória. Enquanto os arrays têm tamanho fixo, os vetores (std::vector) podem crescer dinamicamente.

Funções são blocos de código reutilizáveis que executam uma tarefa específica. Elas podem receber parâmetros e retornar valores, permitindo modularização e organização do código.

Matrizes são arrays bidimensionais (ou de mais dimensões) usados para armazenar dados tabulares. Em C++, são declaradas como `int matriz[3][3]`, representando uma grade de elementos.